```
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ストレス発散 扇風機 & 記憶ゲーム</title>
    <style>
        :root {
            --primary: #2563eb;
            --primary-hover: #1d4ed8;
            --bg-color: #f8fafc;
            --card-bg: #ffffff;
            --text-color: #1e293b;
            --border-color: #cbd5e1;
        }
        body {
            font-family: 'Helvetica Neue', Arial, sans-serif;
            background-color: var(--bg-color);
            color: var(--text-color);
            margin: 0;
            padding: 20px;
            display: flex;
            flex-direction: column;
            align-items: center;
        }
        .container {
            width: 100%;
            max-width: 600px;
            background: var(--card-bg);
            padding: 24px;
            border-radius: 16px;
            box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1);
        }
        h1 {
            font-size: 1.5rem;
            text-align: center;
            margin-bottom: 20px;
        }
        .fan-box {
            text-align: center;
            margin-bottom: 20px;
            position: relative;
            height: 180px;
            background: #e2e8f0;
            border-radius: 12px;
            display: flex;
            align-items: center;
            justify-content: center;
```

```
    overflow: hidden;
}
.fan-blade {
    width: 120px;
    height: 120px;
    border-radius: 50%;
    border: 6px dashed #64748b;
    position: absolute;
    transition: transform 0.1s linear;
}
.spinning {
    animation: spin 1s linear infinite;
}
.fast-spin {
    animation: spin 0.3s linear infinite;
}
@keyframes spin {
    0% { transform: rotate(0deg); }
    100% { transform: rotate(360deg); }
}
.controls {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 8px;
    margin-bottom: 20px;
}
.btn {
    padding: 12px 4px;
    font-size: 0.85rem;
    font-weight: bold;
    border: 2px solid var(--border-color);
    border-radius: 8px;
    background: #f1f5f9;
    cursor: pointer;
    transition: all 0.2s;
    text-align: center;
}
.btn:hover {
    background: #e2e8f0;
}
.btn.active-target {
    background: #fbbf24;
    border-color: #f59e0b;
    box-shadow: 0 0 12px #f59e0b;
    transform: scale(1.05);
}
.game-status {
    text-align: center;
```

```
    font-size: 1.1rem;
    font-weight: bold;
    margin-bottom: 15px;
    min-height: 30px;
    color: #d97706;
}
.ranking-section {
    margin-top: 30px;
    border-top: 2px solid #e2e8f0;
    padding-top: 20px;
}
.tabs {
    display: flex;
    gap: 4px;
    margin-bottom: 12px;
    flex-wrap: wrap;
}
.tab-btn {
    flex: 1;
    padding: 8px 4px;
    font-size: 0.75rem;
    background: #e2e8f0;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}
.tab-btn.active {
    background: var(--primary);
    color: white;
}
.ranking-list {
    list-style: none;
    padding: 0;
    margin: 0;
    max-height: 200px;
    overflow-y: auto;
    border: 1px solid var(--border-color);
    border-radius: 8px;
}
.ranking-item {
    padding: 8px 12px;
    border-bottom: 1px solid #f1f5f9;
    display: flex;
    justify-content: space-between;
    font-size: 0.9rem;
}
.ranking-item:last-child {
```

```
            border-bottom: none;
        }
        .modal {
            display: none;
            position: fixed;
            top: 0; left: 0; width: 100%; height: 100%;
            background: rgba(0,0,0,0.5);
            justify-content: center;
            align-items: center;
        }
        .modal-content {
            background: white;
            padding: 24px;
            border-radius: 12px;
            text-align: center;
            width: 90%;
            max-width: 320px;
        }
        .modal-content input {
            width: 80%;
            padding: 8px;
            margin: 12px 0;
            border: 1px solid var(--border-color);
            border-radius: 4px;
        }
    </style>
</head>
<body>

<div class="container">
    <h1>扇風機 & 記憶ゲーム</h1>

    <div class="fan-box" id="fanBox">
        <div class="fan-blade" id="fanBlade"></div>
        <div id="statusText" style="position: absolute; font-weight: bold; color: #475569;">電源
OFF</div>
    </div>

    <div class="game-status" id="gameStatus">ボタン[3]「遊ぶ」を押してゲームスタート！
</div>

    <!-- 5つの操作ボタン -->
    <div class="controls">
        <button class="btn" id="btn1" onclick="pressButton(1)">① 電源</button>
        <button class="btn" id="btn2" onclick="pressButton(2)">② 発散</button>
        <button class="btn" id="btn3" onclick="pressButton(3)">③ 遊ぶ</button>
        <button class="btn" id="btn4" onclick="pressButton(4)">④ 風量</button>
        <button class="btn" id="btn5" onclick="pressButton(5)">⑤ 首振</button>
```

```
    </div>

    <!-- ランキングセクション -->
    <div class="ranking-section">
        <h3>ランキング</h3>
        <div class="tabs">
            <button class="tab-btn active" onclick="switchTab('today')">今日</button>
            <button class="tab-btn" onclick="switchTab('week')">今週</button>
            <button class="tab-btn" onclick="switchTab('month')">今月</button>
            <button class="tab-btn" onclick="switchTab('year')">今年</button>
            <button class="tab-btn" onclick="switchTab('all')">総合 (100位)</button>
        </div>
        <ul class="ranking-list" id="rankingList">
            <li class="ranking-item" style="justify-content: center; color: #94a3b8;">まだ誰もいま
せん</li>
        </ul>
    </div>
</div>

<!-- スコア登録モーダル -->
<div class="modal" id="scoreModal">
    <div class="modal-content">
        <h3>ゲーム終了！</h3>
        <p>スコア: <span id="finalScore">0</span></p>
        <input type="text" id="playerName" placeholder="名前を入力（例: プレイヤー）"
maxlength="15">
        <br>
        <button class="btn" style="background: var(--primary); color: white; width: 100%;"
onclick="saveScore()">ランキングに登録</button>
    </div>
</div>

<script>
    let isPowerOn = false;
    let fanSpeed = 1; // 1:弱, 2:強
    let gameState = 'idle'; // idle, waiting, playing, gameover
    let targetButton = null;
    let score = 0;
    let currentTab = 'today';

    // まだ誰もいない状態のストレージ初期化（空配列）
    const periods = ['today', 'week', 'month', 'year', 'all'];
    periods.forEach(p => {
        if (!localStorage.getItem('ranking_' + p)) {
            localStorage.setItem('ranking_' + p, JSON.stringify([]));
        }
    });
```

```
    function pressButton(num) {
        const statusText = document.getElementById('statusText');
        const fanBlade = document.getElementById('fanBlade');

        // ゲーム中の判定
        if (gameState === 'waiting') {
            // 光る前に押しちゃった場合（フライング・ゲームオーバー）
            triggerGameOver("光る前に押してしまいました！ゲームオーバー！");
            return;
        } else if (gameState === 'playing') {
            if (num === targetButton) {
                // 正解！
                score += 100;
                document.getElementById('gameStatus').innerText = `正解！ スコア: ${score}`;
                clearTimeout(window.gameTimer);
                nextGameRound();
            } else {
                // 間違えたボタンを押した場合
                triggerGameOver("違うボタンを押しました！ゲームオーバー！");
            }
            return;
        }

        // 通常時のボタン機能
        if (num === 1) {
            isPowerOn = !isPowerOn;
            if (isPowerOn) {
                statusText.innerText = fanSpeed === 2 ? "電源ON (強)" : "電源ON (弱)";
                fanBlade.className = fanSpeed === 2 ? "fan-blade fast-spin" : "fan-blade
spinning";
            } else {
                statusText.innerText = "電源OFF";
                fanBlade.className = "fan-blade";
            }
        } else if (num === 2) {
            // ストレス発散アクション
            statusText.innerText = "💥 ストレス発散！ビューーーッ！";
            setTimeout(() => {
                statusText.innerText = isPowerOn ? (fanSpeed === 2 ? "電源ON (強)" : "電源ON
(弱)") : "電源OFF";
            }, 1000);
        } else if (num === 3) {
            // 遊ぶ（ゲーム開始）
            startMemoryGame();
        } else if (num === 4) {
            if (!isPowerOn) return;
            fanSpeed = fanSpeed === 1 ? 2 : 1;
            statusText.innerText = fanSpeed === 2 ? "電源ON (強)" : "電源ON (弱)";
```

```
            fanBlade.className = fanSpeed === 2 ? "fan-blade fast-spin" : "fan-blade spinning";
        } else if (num === 5) {
            statusText.innerText = "🔄 首振り中...";
            setTimeout(() => {
                statusText.innerText = isPowerOn ? (fanSpeed === 2 ? "電源ON (強)" : "電源ON
(弱)") : "電源OFF";
            }, 1000);
        }
    }

    function startMemoryGame() {
        score = 0;
        gameState = 'waiting';
        document.getElementById('gameStatus').innerText = "まもなくボタンが光ります…（フライ
ング注意！）";

        // 1.5秒〜3秒後にランダムなボタン（1〜5）が光る
        const randomTime = 1500 + Math.random() * 1500;
        window.gameTimer = setTimeout(() => {
            if (gameState !== 'waiting') return;
            targetButton = Math.floor(Math.random() * 5) + 1;
            gameState = 'playing';

            document.getElementById('gameStatus').innerText = `ボタン[${targetButton}]が光っ
た！今すぐ押せ！`;
            const btnEl = document.getElementById(`btn${targetButton}`);
            btnEl.classList.add('active-target');

            // 1.2秒以内に押さないとタイムアウト
            window.timeoutTimer = setTimeout(() => {
                if (gameState === 'playing') {
                    triggerGameOver("タイムアウト！ゲームオーバー！");
                }
            }, 1200);
        }, randomTime);
    }

    function nextGameRound() {
        // ボタンの光るクラスをリセット
        for (let i = 1; i <= 5; i++) {
            document.getElementById(`btn${i}`).classList.remove('active-target');
        }
        clearTimeout(window.timeoutTimer);

        // 次のラウンドへ（少し早めに再スタート）
        setTimeout(() => {
            if (gameState === 'gameover') return;
            gameState = 'waiting';
```

```
        document.getElementById('gameStatus').innerText = "次も光るのを待て…";
        const randomTime = 1000 + Math.random() * 1500;
        window.gameTimer = setTimeout(() => {
            if (gameState !== 'waiting') return;
            targetButton = Math.floor(Math.random() * 5) + 1;
            gameState = 'playing';
            document.getElementById('gameStatus').innerText = `ボタン[${targetButton}]が
光った！今すぐ押せ！`;
            document.getElementById(`btn${targetButton}`).classList.add('active-target');

            window.timeoutTimer = setTimeout(() => {
                if (gameState === 'playing') {
                    triggerGameOver("タイムアウト！ゲームオーバー！");
                }
            }, 1000 - Math.min(score, 500)); // スコアが上がると制限時間が短くなる
        }, randomTime);
    }, 500);
}

function triggerGameOver(msg) {
    gameState = 'gameover';
    clearTimeout(window.gameTimer);
    clearTimeout(window.timeoutTimer);
    for (let i = 1; i <= 5; i++) {
        document.getElementById(`btn${i}`).classList.remove('active-target');
    }
    document.getElementById('gameStatus').innerText = msg;

    // モーダル表示
    document.getElementById('finalScore').innerText = score;
    document.getElementById('scoreModal').style.display = 'flex';
}

function saveScore() {
    const nameInput = document.getElementById('playerName').value.trim() || "名無しさん";

    periods.forEach(p => {
        let list = JSON.parse(localStorage.getItem('ranking_' + p)) || [];
        list.push({ name: nameInput, score: score, date: new Date().toLocaleDateString() });
        // スコア順に並び替え
        list.sort((a, b) => b.score - a.score);

        // 総合は100位まで、他は1000位まで
        const maxLimit = p === 'all' ? 100 : 1000;
        if (list.length > maxLimit) {
            list = list.slice(0, maxLimit);
        }
        localStorage.setItem('ranking_' + p, JSON.stringify(list));
```

```
        });

        document.getElementById('scoreModal').style.display = 'none';
        document.getElementById('gameStatus').innerText = "スコアが保存されました！ボタン
[3]で再挑戦";
        switchTab(currentTab);
    }

    function switchTab(period) {
        currentTab = period;
        document.querySelectorAll('.tab-btn').forEach(btn => btn.classList.remove('active'));
        event && event.target.classList.add('active');

        const listEl = document.getElementById('rankingList');
        const data = JSON.parse(localStorage.getItem('ranking_' + period)) || [];

        if (data.length === 0) {
            listEl.innerHTML = `<li class="ranking-item" style="justify-content: center; color:
#94a3b8;">まだ誰もいません</li>`;
            return;
        }

        let html = '';
        data.forEach((item, index) => {
            html += `<li class="ranking-item">
                <span>${index + 1}位: ${item.name}</span>
                <strong>${item.score}点</strong>
            </li>`;
        });
        listEl.innerHTML = html;
    }

    // 初期化時にランキング表示を更新
    switchTab('today');
</script>

</body>
</html>
```